



南開大學
Nankai University

计算机学院

深度学习及应用（高阶课）实验报告

循环神经网络算法

姓名：申健强

学号：2313119

专业：计算机科学与技术

2026 年 6 月 29 日

目录

1 实验目的与要求	2
2 数据集与实验设置	2
3 模型结构与原理	2
3.1 RNN	2
3.2 LSTM	3
4 实验结果	3
5 讨论：为什么 LSTM 优于 RNN	4
6 扩展：基于字符级 RNN 的名字生成	6
7 遇到的问题与解决方法	7
8 结论	7

1 实验目的与要求

本实验的目标是掌握循环神经网络（RNN）的基本原理，并使用 PyTorch 在名字识别任务（根据一个姓名预测其所属的语言/国家，共 18 类）上完成以下工作：

- 复现老师提供的原始版本 RNN，给出网络结构、在验证集上的训练 loss 曲线、准确率曲线以及预测（混淆）矩阵图；
- 自行实现 LSTM 网络完成同样的任务，给出其结构、loss 曲线、准确率曲线与混淆矩阵图；
- 重点分析为什么 LSTM 的性能优于 RNN；

作为扩展，我们另外实现了基于字符级 RNN 的名字生成任务，单列一节讨论（见第 6 节）。

2 数据集与实验设置

数据集. 采用 PyTorch 官方提供的 names 数据集，包含 18 种语言、共 20,074 个姓名，每种语言一个文本文件。每个名字先经 Unicode 转 ASCII 规范化，再表示为字符 one-hot 序列：字符表为大小写字母加 " .,;'" 与一个 OOV 占位符，共 `n_letters=58` 个，于是一个长度为 L 的名字被编码为形状 $(L, 1, 58)$ 的张量。按 85%/15% 随机划分为训练集 (17,063) 与验证集 (3,011)，随机种子固定为 2024。

训练协议. 两个模型使用完全相同的设置以保证可比：损失函数为负对数似然 `NLLLoss`（配合输出端 `LogSoftmax`），优化器为 SGD（学习率 0.15）。训练沿用 notebook 的写法——按“伪 batch”（每 64 个样本）累加单样本损失后再统一反向、做梯度裁剪（阈值 3）并更新一次参数；隐藏层维度 128，训练 15 个 epoch。我们在每个 epoch 结束后于验证集上评估一次 Top-1 准确率并记录，用于绘制准确率曲线。模型定义在 `models.py`，数据/训练/评估工具在 `train_utils.py`，可视化在 notebook 中完成直观展示实验结果。

3 模型结构与原理

3.1 RNN

基线使用 PyTorch 内置的 `nn.RNN` 逐字符读入名字，取最后时刻的隐藏态经线性层与 `LogSoftmax` 得到 18 类的对数概率。普通 RNN 的隐藏态按下式递推：

$$\mathbf{h}_t = \tanh(W_{ih} \mathbf{x}_t + W_{hh} \mathbf{h}_{t-1} + \mathbf{b}). \quad (1)$$

其 `print(net)` 输出如下；前向时取最后时刻隐藏态 `hidden[0]` 过 `h2o` 与 `LogSoftmax` 得到 18 类对数概率：

```
1 NameRNN(  
2   (rnn): RNN(58, 128)  
3   (h2o): Linear(in_features=128, out_features=18, bias=True)  
4   (softmax): LogSoftmax(dim=1)  
5 )
```

3.2 LSTM

我们手动实现了 LSTM，不调用 `nn.LSTM`。在时刻 t ，给定输入 $\mathbf{x}_t$ 、上一隐藏态 $\mathbf{h}_{t-1}$ 与上一细胞态 $\mathbf{c}_{t-1}$ ：

$$\begin{aligned} \mathbf{i}_t &= \sigma(W_i[\mathbf{x}_t, \mathbf{h}_{t-1}]), & \mathbf{f}_t &= \sigma(W_f[\mathbf{x}_t, \mathbf{h}_{t-1}]), \\ \mathbf{g}_t &= \tanh(W_g[\mathbf{x}_t, \mathbf{h}_{t-1}]), & \mathbf{o}_t &= \sigma(W_o[\mathbf{x}_t, \mathbf{h}_{t-1}]), \\ \mathbf{c}_t &= \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \mathbf{g}_t, & \mathbf{h}_t &= \mathbf{o}_t \odot \tanh(\mathbf{c}_t). \end{aligned} \quad (2)$$

其中 $\mathbf{i}, \mathbf{f}, \mathbf{o}$ 分别为输入门、遗忘门、输出门， $\mathbf{c}_t$ 为细胞态。四个门的线性变换合并为一个 `Linear(186, 512)` 后切分。`print(net)` 显示其注册子模块为合并四门的 `x2h` 与输出层 `h2o`，四个门的循环计算在手写 `forward` 中完成：

```
1 NameLSTM(                                     # print(net): registered submodules
2   (x2h): Linear(in_features=186, out_features=512, bias=True) # combined i,f,g,o gates
3   (h2o): Linear(in_features=128, out_features=18, bias=True)
4   (softmax): LogSoftmax(dim=1)
5 )
6 def forward(self, line_tensor):                # hand-written gating (bonus), no nn.LSTM
7   h = torch.zeros(1, hidden); c = torch.zeros(1, hidden)
8   for t in range(line_tensor.size(0)):
9     i, f, g, o = self.x2h(torch.cat([line_tensor[t], h], 1)).chunk(4, 1)
10    i, f, g, o = i.sigmoid(), f.sigmoid(), g.tanh(), o.sigmoid()
11    c = f * c + i * g                            # cell state
12    h = o * torch.tanh(c)                        # hidden state
13  return self.softmax(self.h2o(h))
```

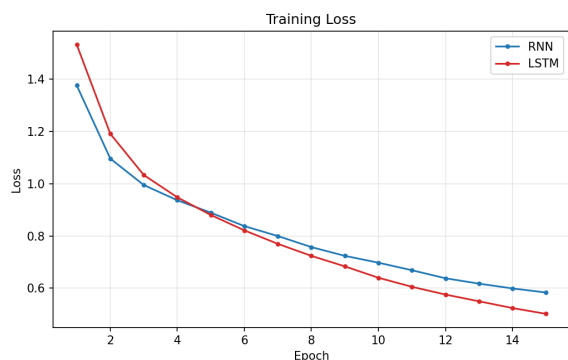
4 实验结果

两个模型在相同协议下各训练 15 个 epoch，总体结果如表 1 所示。手写 LSTM 取得 **81.20%** 的验证准确率，高于 RNN 的 78.74%。

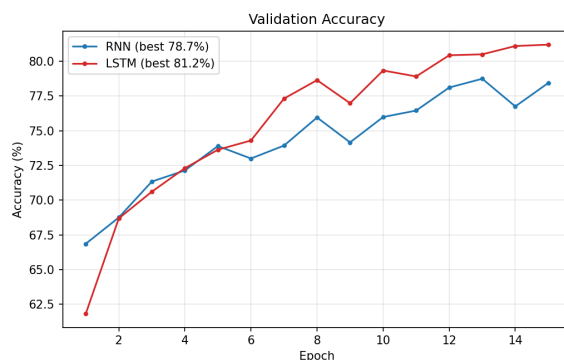
表 1: RNN 与 LSTM 总体结果对比

模型	参数量	最佳准确率 (%)	最终准确率 (%)	训练耗时 (min)
RNN (基线)	26,386	78.74	78.45	9.2
LSTM (手写, 加分)	98,066	81.20	81.20	18.5

图 4.1 为两个模型的训练 loss 曲线与验证准确率曲线（同坐标系便于对比）。可见 LSTM 的 loss 下降更充分、验证准确率曲线整体在 RNN 之上，且末段更平稳。



(a) 训练 loss 曲线 (RNN vs LSTM)



(b) 验证准确率曲线 (RNN vs LSTM)

图 4.1: RNN 与 LSTM 的训练 loss 与验证准确率曲线对比

图 4.2 为两个模型在验证集上的预测（混淆）矩阵（按真实类别行归一化，对角线越亮表示该类识别越准）。

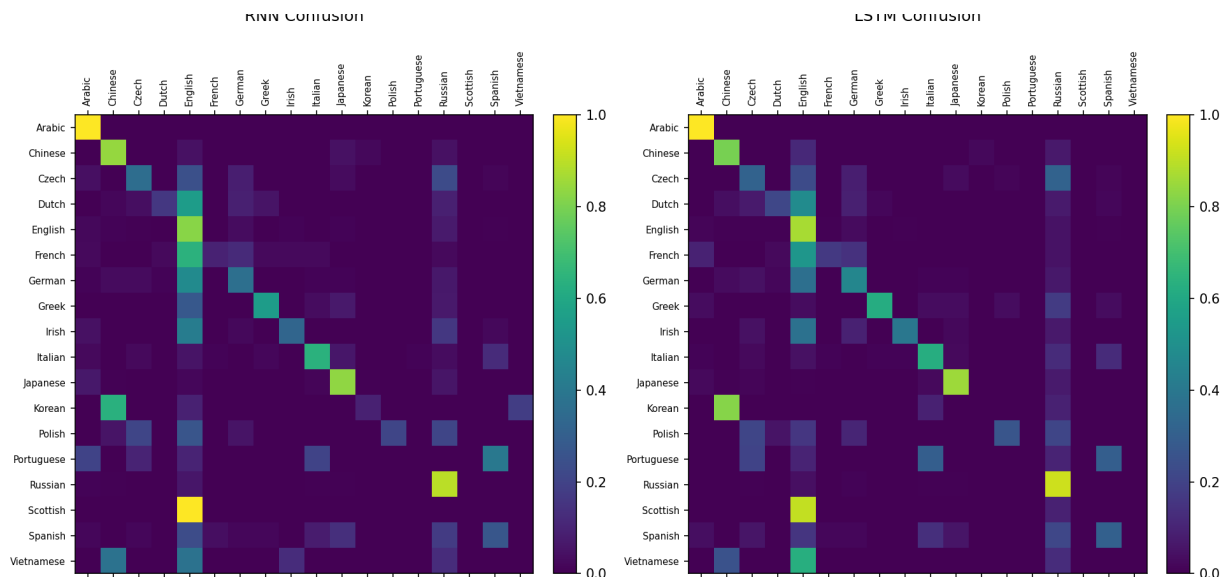


图 4.2: 预测（混淆）矩阵：左 RNN，右 LSTM

逐类别观察。从混淆矩阵的对角线（每类召回率）看，LSTM 在多数类别上优于 RNN，提升最明显的有 German (+9.2)、French (+7.1)、Irish (+7.0)、Greek (+6.9)、Dutch (+5.4)、Polish (+5.3)、English (+4.7) 等；仅在 Chinese、Czech、Korean 等少数类别略有下降。此外，数据集类别极不均衡（如 Russian、English、Arabic、Japanese 样本众多，而 Portuguese、Scottish、Vietnamese 等样本很少），导致这些小类两个模型的召回率都接近 0，模型倾向于把它们误判为字符风格相近的大类——这也是整体准确率高于各类别平均准确率的原因。

5 讨论：为什么 LSTM 优于 RNN

普通 RNN 仅靠单一隐藏态 h_t 在时间维上反复经非线性与矩阵相乘传播信息。在反向传播时，梯度需连乘多个雅可比矩阵，序列略长便容易出现**梯度消失或爆炸**，使网络难以学习较早字符与类别之间的依赖；对名字识别而言，词尾、词中的关键字符信息在长名字里难以稳定保留。

LSTM 通过引入细胞态 c_t 和输入门/遗忘门/输出门缓解了这一问题：

- 细胞态以 $c_t = f_t \odot c_{t-1} + i_t \odot g_t$ 的加性方式更新，当遗忘门 $f_t \approx 1$ 时梯度可近似线性地沿 c_t 传播，从而缓解梯度消失，使长程依赖得以保留；
- 三个门是数据相关的开关：输入门控制写入多少新信息，遗忘门控制保留多少历史，输出门控制暴露多少细胞态，赋予网络选择性记忆的能力。

机制与行为验证. 我们从两个角度给出可视化证据进行验证和分析（图 5.3）。其一（左图，梯度沿时间反传）：在随机初始化下度量输出对各时间步隐藏态的梯度 $\|\partial h_T / \partial h_t\|$ ，普通 RNN 随回溯距离指数衰减（回溯 18 步后降到约 10^{-4} ）；而 LSTM 只有在遗忘门打开（ $f \approx 1$ ）时细胞态才成为“梯度高速路”、衰减最慢（比同距离的 RNN 高约 70 倍），若遗忘门接近 0.5 则衰减反而比 RNN 更快——这定量地印证了“加性细胞态在 $f \approx 1$ 时缓解梯度消失”，也解释了为何遗忘门偏置常初始化为 1。其二（右图，真实任务行为）：随名字长度增加，LSTM 相对 RNN 的准确率优势从短名字的约 +1.9 个百分点扩大到 11 字符以上的约 +4.3 个百分点，说明优势确来自对更长上下文的更好保持。但值得注意的是：本任务名字较短（多为 5 ~ 10 字符），梯度消失并非 LSTM 取胜的唯一原因，门控带来的更强表达能力同样重要。

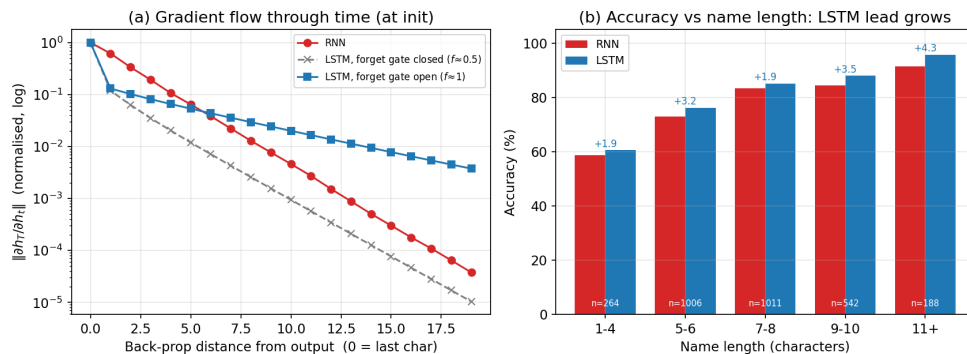


图 5.3: 为什么 LSTM 优于 RNN。左：初始化时梯度沿时间反传的幅值 $\|\partial h_T / \partial h_t\|$ （对数纵轴）——RNN 指数消失，LSTM 仅在遗忘门打开时保持梯度（遗忘门关闭时甚至更差）。右：真实任务上准确率随名字长度的变化，LSTM 的领先幅度随长度增大（+1.9 → +4.3 pp）。

逐字符的读懂过程可视化. 图 5.4 把名字逐字符喂入分类器，展示预测语言的概率如何随每个字符演化（行 = 候选语言，列 = 已读字符，绿框 = 真实语言）。可见 LSTM 往往能在读完整个名字后更果断地“锁定”正确语言（真类一行末尾显著变亮），而 RNN 易在中途被局部字符组合带偏（如读到“Majew”时一度高概率判成 Japanese），最终把握也更弱。这与“门控带来的选择性记忆使其更稳地整合整段序列信息”一致。

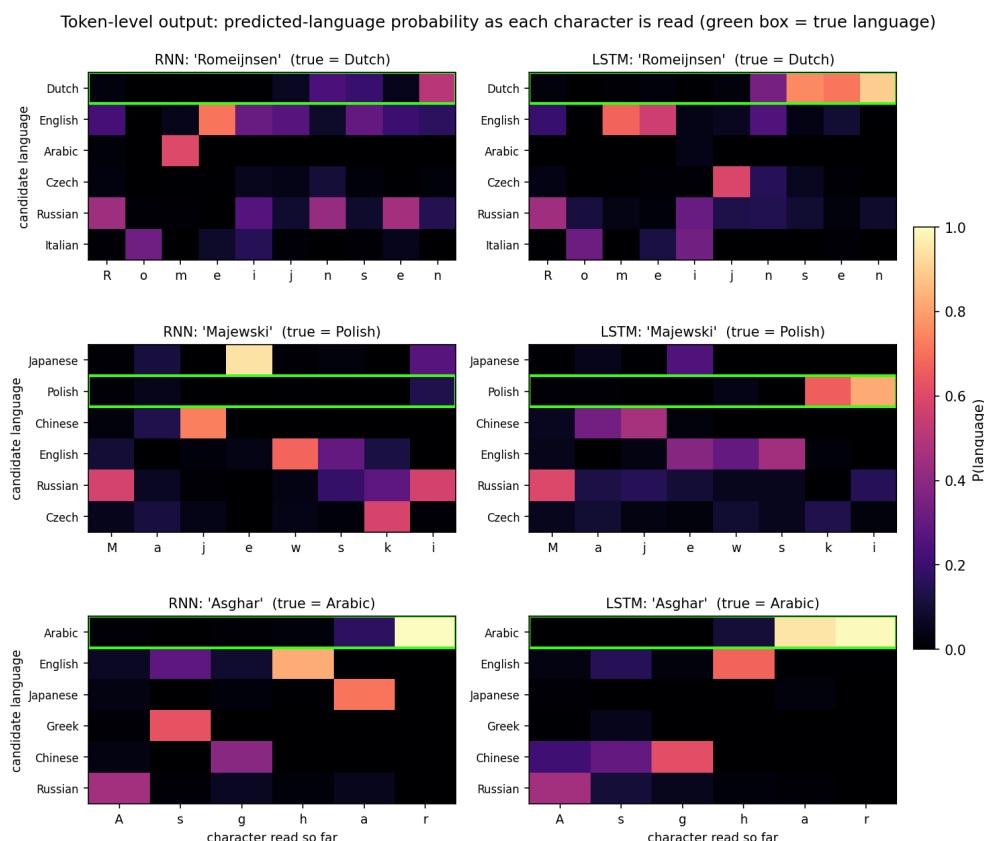


图 5.4: Token 级输出：逐字符读入时预测语言的概率演化（绿框 = 真实语言）。LSTM 末端对真类更果断，RNN 中途易被带偏。

这与实验结果一致：LSTM 的验证准确率（81.20%）高于 RNN（78.74%），收敛更稳，且在需要更长上下文才能区分的语言（如 German、French、Irish、Greek 等）上召回率提升显著（见上节逐类别观察）。代价是 LSTM 参数量（约 98K）与训练时间（18.5 min）大于 RNN（26K，9.2 min），属于以更高计算开销换取更强的序列建模能力。

6 扩展：基于字符级 RNN 的名字生成

除分类外，我们实现了**名字生成任务**：给定类别（语言）和起始字母，让网络逐字符地预测下一个字符，直到生成结束符 EOS。与分类只用最后一步输出不同，生成在**每一步**都做预测并累加损失。网络在每个时间步的输入为“类别 one-hot + 当前字符 one-hot + 上一隐藏态”的拼接，结构如下：

```

1 class RNN(nn.Module):                                # category-conditioned char generator
2     def __init__(self, input_size, hidden_size, output_size):
3         super().__init__()
4         self.hidden_size = hidden_size
5         self.i2h = nn.Linear(n_categories + input_size + hidden_size, hidden_size)
6         self.i2o = nn.Linear(n_categories + input_size + hidden_size, output_size)
7         self.o2o = nn.Linear(hidden_size + output_size, output_size)
8         self.dropout = nn.Dropout(0.1)
9         self.softmax = nn.LogSoftmax(dim=1)
10    def forward(self, category, input, hidden):
11        combined = torch.cat((category, input, hidden), 1)
12        hidden = self.i2h(combined)
13        output = self.o2o(torch.cat((hidden, self.i2o(combined)), 1))

```

```
14 return self.softmax(self.dropout(output)), hidden
```

训练时对每个名字，把相邻字符构成 (当前字符 → 下一字符) 的监督对，按时间步累加 `NLLLoss` 后整体反向。

采样策略的影响。最初采用**贪心解码** (每步取概率最大的字符)，存在两个问题：一是对同一 (类别, 起始字母) 输出完全确定、毫无多样性；二是容易卡在高频字符组合上重复，例如俄语名字会出现 `Uanovovov` 这类重复尾巴。改为**按概率分布采样并引入温度系数 T** 后多样性明显改善：

$$P'(\text{char}) \propto P(\text{char})^{1/T}, \quad (3)$$

T 越小越保守 (接近贪心)，越大越随机；名字生成取 $T \approx 0.5 \sim 0.8$ 时既保持各语言风格又具有多样性。该实验说明：生成质量不仅取决于模型，也显著依赖解码/采样策略。

7 遇到的问题与解决方法

实验过程中的问题主要集中在手写 LSTM 的正确性、训练稳定性和数据特性三方面。

(1) 手写 LSTM 初期 loss 几乎不下降。现象是 LSTM 训练前几个 epoch 准确率停在随机水平。定位时把四个门的输出打印出来，发现某次实现里把 `gates.chunk(4)` 得到的四份错配成了 `i, g, f, o` 的顺序，导致“候选记忆 g ”被加了 sigmoid、而“遗忘门 f ”过了 tanh，门控语义完全错乱。根因是 `chunk` 的切分顺序必须与“`i, f, g, o` 各自应用 sigmoid/tanh”严格对应。修复后 (`i, f, o` 用 sigmoid、`g` 用 tanh, $c_t = f \odot c_{t-1} + i \odot g$) loss 正常下降，验证准确率回到 80% 量级，说明门控单元实现正确。

(2) 训练中损失偶发性突跳。个别迭代 loss 会突然放大。考虑到 RNN/LSTM 在按时间展开后等价于很深的网络、长名字处梯度连乘容易放大，判断是梯度爆炸所致。解决方法是沿用老师写法在反向后、更新前加入梯度裁剪 `clip_grad_norm_(params, 3.0)`；裁剪后损失曲线平滑、不再出现突跳，两个模型在同一裁剪阈值下对比也更公平。

(3) 部分小语种召回率始终接近 0。混淆矩阵显示 Portuguese、Scottish、Vietnamese 等类别几乎全被误判。检查数据后确认根因是**类别极度不平衡**：这些语言的样本数仅为 Russian、English 等大类的百分之一量级，模型在交叉熵下倾向于多数类。我们据此明确了评价口径——以**整体准确率**为主指标的同时，必须结合**逐类召回** (混淆矩阵，图 4.2) 来识别这种“整体高、小类塌陷”的现象，避免只看总准确率而高估模型；若要真正改善小类，可引入类别加权采样或加权损失，这也是后续可改进的方向。此外还需保证训练与推理使用**同一字符表** (含 OOV 占位符, `n_letters=58`)，否则 one-hot 索引错位会直接破坏输入语义。

8 结论

本实验在名字识别任务上对比了 RNN 与手写实现的 LSTM。在实验设置上，LSTM 以 81.20% 的验证准确率明显优于 RNN 的 78.74%，收敛更加稳定、在多数语言类别上召回率更高；对比分析可以发现原因在于细胞态的加性更新与门控机制有效缓解了梯度消失、增强了长程依赖建模，不过代价是更大的参数量与训练时间。

参考文献

[1] J. L. Elman. Finding Structure in Time. *Cognitive Science*, 14(2):179–211, 1990.

- [2] S. Hochreiter, J. Schmidhuber. Long Short-Term Memory. *Neural Computation*, 9(8):1735–1780, 1997.
- [3] A. Paszke et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library. *NeurIPS*, 2019.
- [4] S. Robertson. NLP From Scratch: Classifying / Generating Names with a Character-Level RNN. PyTorch Tutorials.